

Lab 3: Architectural Justification

Problem Statement #41 – Open API Mock Server Generator

Name: T Surya

SRN: PES1UG24AM300

Architecture Selection : We chose Microservices Architecture for the OpenAPI Mock Server Generator.

Architectural Choice : A microservices-based structure was selected with five services, matching the component diagram. The OpenAPI Specification Parser accepts the OpenAPI 3.0 YAML/JSON specification and exposes a Parsed OpenAPI Definition; the Mock Endpoint Generator consumes that definition to build dynamic mock endpoints, and calls the Schema Validator through a Schema Validation interface to obtain a Validation Result; once validated, the Mock Endpoint Generator requests a response through the Generate Mock Response interface provided by the Mock Response Generator; the generated payload is then passed through the Apply Response Latency interface to the Latency Simulator, which returns the final Mock HTTP Response to the client. Each service is independently deployable and communicates with the others through these well-defined interfaces, so any service under heavy load can be scaled independently without scaling the rest of the system.

Two Scenario-Related Reasons

- **Independent Scalability:** The system must sustain 1,000 mock API requests per second. Microservices allow the individual components that handle API requests and generate responses to be scaled independently based on their workload, rather than scaling the entire system to meet the demand of its busiest part.
- **Separation of Responsibilities:** The system performs several distinct functions, including OpenAPI specification parsing, dynamic mock endpoint generation, schema validation, mock response generation, and response latency simulation. Separating these functions into independent services gives each one a clear, single responsibility and makes the system easier to develop, test, and maintain.

Security Advantage

The Schema Validator service can reject malformed or invalid API requests before they are processed further by any other service. This supports the security requirement that invalid requests must be rejected without exposing sensitive server information, since the rejection happens at a single, well-defined boundary rather than being handled inconsistently across the system.

Performance Benefit

Microservices allow the most heavily used components of the mock server, such as the Mock Endpoint Generator and Mock Response Generator, to be scaled independently, helping the system achieve the required throughput of 1,000 requests per second. The Latency Simulator can apply configurable response latency through the Apply Response Latency interface without affecting the performance of the other services, allowing the system to simulate different API response conditions on demand before returning the final Mock HTTP Response.

OpenAPI Mock Server Generator

